

CaptionAI: AI-Powered Social Media Caption Generator

Project Description:

Project Name: CaptionAI — AI-Powered Social Media Content Generator

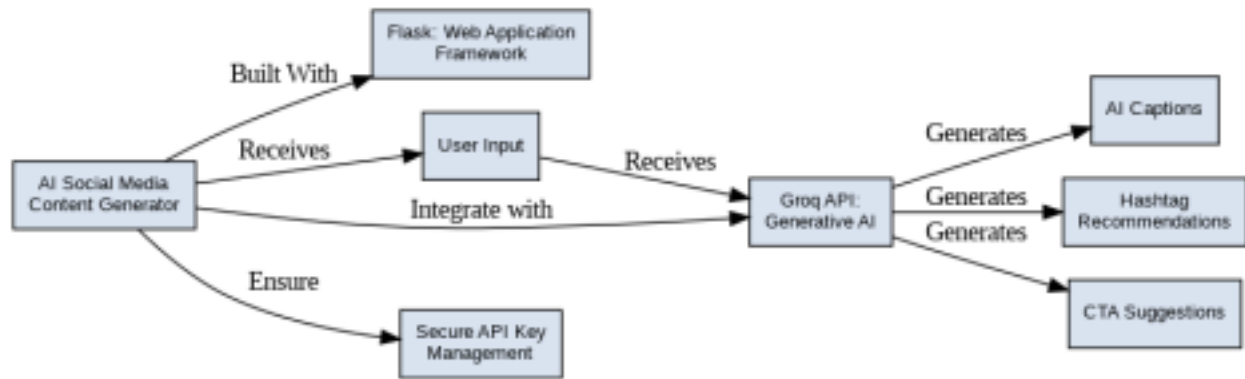
Domain: Generative Artificial Intelligence / Web Development / Social Media Automation

Core Objective: CaptionAI is an intelligent, full-stack web application designed to automate and streamline social media copywriting for marketers, content creators, brand managers, and digital agencies. By leveraging high-speed Large Language Models (LLMs), the platform transforms raw product or campaign descriptions into platform-optimized social media content kits within seconds.

Scenarios

- **Scenario 1 (Instagram / Casual):**
 - *Input:* New summer collection of eco-friendly bamboo sunglasses.
 - *Output:* Lighthearted, emoji-rich storytelling captions, mixed environmental/fashion tags (**#EcoFashion**, **#BambooSunglasses**), and friendly engagement CTAs.
- **Scenario 2 (LinkedIn / Professional):**
 - *Input:* Automated B2B payroll management software launch.
 - *Output:* Value-driven, metric-oriented copy highlighting ROI, B2B tags (**#HRTech**, **#SaaS**), and conversion CTAs ("Book a live demo today").
- **Scenario 3 (Instagram / Promotional):**
 - *Input:* 48-hour weekend flash sale on fashion apparel.
 - *Output:* High-urgency copy with power words ("50% Off", "Limited Stock"), deal-seeker tags (**#FlashSale**, **#WeekendDeals**), and direct action CTAs ("Shop before it's gone!").

Technical Architecture:



Presentation Layer (Frontend):

- Built with HTML5, CSS3 (Glassmorphism design), and Vanilla JavaScript.
- Captures user input (product description, platform, tone) and sends asynchronous **HTTP POST** requests via the Fetch API.
- Dynamically renders captions with copy buttons, hashtag chips, and CTAs.

Application Layer (Backend):

- Built with Python & Flask (**app.py**).
- Validates inputs (checks for empty brief and max 2,000 characters).
- Builds prompts using platform guidelines and tone parameters.
- Parses output (**extract_json**) using fallback regex to guarantee valid JSON formatting.
- Loads secrets securely from a **.env** file via **python-dotenv**.

Inference Layer (AI Core):

- Powered by Groq's LLaMA 3.3-70B Versatile model.
- Configured with **temperature=0.8** and **max_tokens=2048**.
- Strictly enforces raw JSON responses containing 3 captions, 10 hashtags, and 3 CTAs.

Deployment & Tunneling:

- Runs locally on Flask (port 5050).
- Exposed publicly using Ngrok (**pyngrok**) via **run_public.py** for instant remote testing.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- Flask Framework Knowledge: [Flask Documentation](#)

- **Groq API Familiarity:** [Groq API Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#) •
- Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)
- **Ngrok for Public Deployment:** [Ngrok Documentation](#)



Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment. •
- Activity 1.2:** Research and select the appropriate generative AI model from Groq for social media content generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask and the Groq API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Caption Generator, Hashtag Suggester, and Call-to-Action Generator.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth API interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 6: Conclusion



Milestone 1: Model Selection and

Milestone Objective

Establish the project foundation by securing API credentials, evaluating and selecting the optimal Generative AI model from Groq, defining the complete technical architecture, and setting up an isolated development environment.

Activity 1.1: Generate and Secure Groq API Key

To enable communication between the backend and Groq's LLM infrastructure:

1. **Account Setup:** Registered on the Groq Console (<https://console.groq.com>).
2. **Key Generation:** Generated a new API key named `captionai-key`.
3. **Local Storage:** Configured the API key inside a local `.env` file in the project root:
4. Code snippet

```
GROQ_API_KEY=gsk_your_actual_api_key_here
```

- 5.
- 6.
7. **Environment Isolation:** Loaded keys into `app.py` using `python-dotenv`:
8. Python

```
from dotenv import load_dotenv
```

```
import os, groq
```

```
load_dotenv()
```

```
client = groq.Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

- 9.
- 10.

11. **Security Practice:** Added `.env` to `.gitignore` to prevent credential exposure in version control.

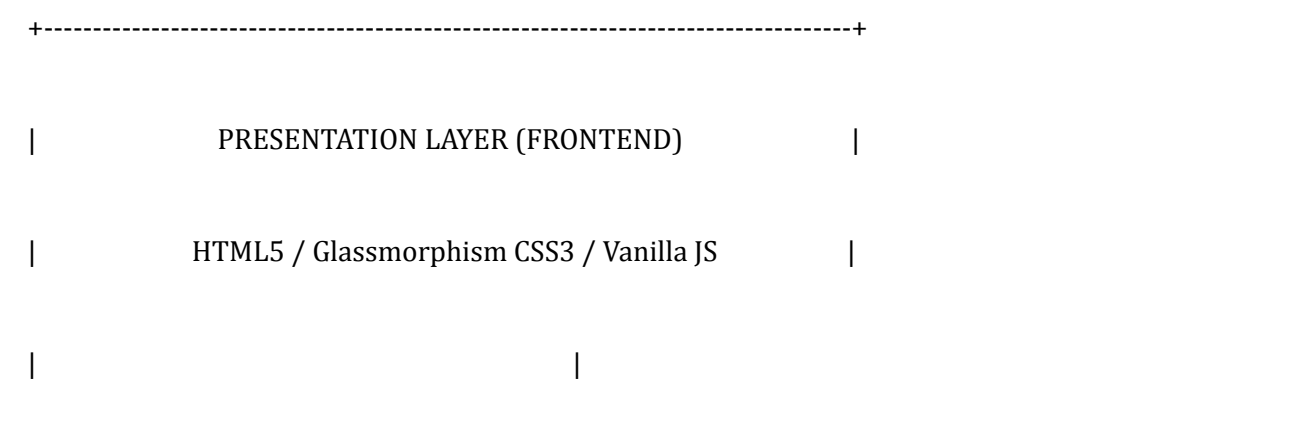
Activity 1.2: Research and Model Selection

Evaluated Groq's available LLM offerings against project requirements (generating high-converting social media captions, balanced hashtags, and CTAs across various platforms and tones):

Criterion	Requirement	Model Choice: llama-3.3-70b-versatile
Response Speed	Instant output for seamless UI experience	Ultra-fast inference leveraging Groq LPUs
Copywriting Quality	Platform-tailored, engaging, and tone-specific	High creative reasoning and contextual understanding
JSON Formatting	Strict adherence to structured JSON schemas	Strong instruction-following capability for schema enforcement
Configuration	Balanced creativity without hallucinations	temperature = 0.8

Activity 1.3: Define Application Architecture

Designed a decoupled **Three-Tier Architecture** separating the UI, business logic, and AI core:



| Inputs: Product Details (Max 2000 chars) | Platform Toggle | Tone Selection |

+-----+-----+

| HTTP POST /generate (JSON)

v

+-----+

| APPLICATION LAYER (BACKEND WSGI) |

| Flask Framework |

| |

| • Input Validation & Sanitization |

| • Dynamic Prompt Engineering (build_prompt()) |

| • JSON Parsing & Fallback Extraction (extract_json()) |

+-----+-----+

| Encrypted API Request (Groq SDK)

v

+-----+

| INFERENCE LAYER (AI CORE) |

	Groq Cloud (LLaMA 3.3-70B Versatile)	
	• Outputs strictly structured JSON (3 Captions, 10 Hashtags, 3 CTAs)	

+-----+

Component Roles:

- **Frontend:** Interactive glassmorphism single-page UI capturing user inputs and rendering outputs asynchronously via **Fetch API**.
- **Backend:** Flask web server handling routing (**/** and **/generate**), parameter validation, prompt assembly, and error handling.
- **AI Integration:** Passes engineered system and user prompts to Groq API and parses the returned text into structured JSON.

Activity 1.4: Set Up Development Environment

Configured the local workspace and installed required dependencies:

1. **Virtual Environment Creation:**
2. Bash

```
python -m venv myenv
```

Windows activation:

```
myenv\Scripts\activate
```

macOS/Linux activation:

```
source myenv/bin/activate
```

- 3.
- 4.

5. **Dependency Installation (`requirements.txt`):**

6. Plaintext

Flask==3.0.3

groq==0.11.0

python-dotenv==1.0.1

pyngrok==7.1.6

7.

8. Bash

pip install -r requirements.txt

9.

10.

11. **Directory Structure Setup:**

12. Plaintext

captionai/

├── static/

| ├── css/style.css

| └── js/main.js

├── templates/

| └── index.html

├── .env

└── .gitignore

|— app.py

|— requirements.txt

|— run_public.py

13.

14.

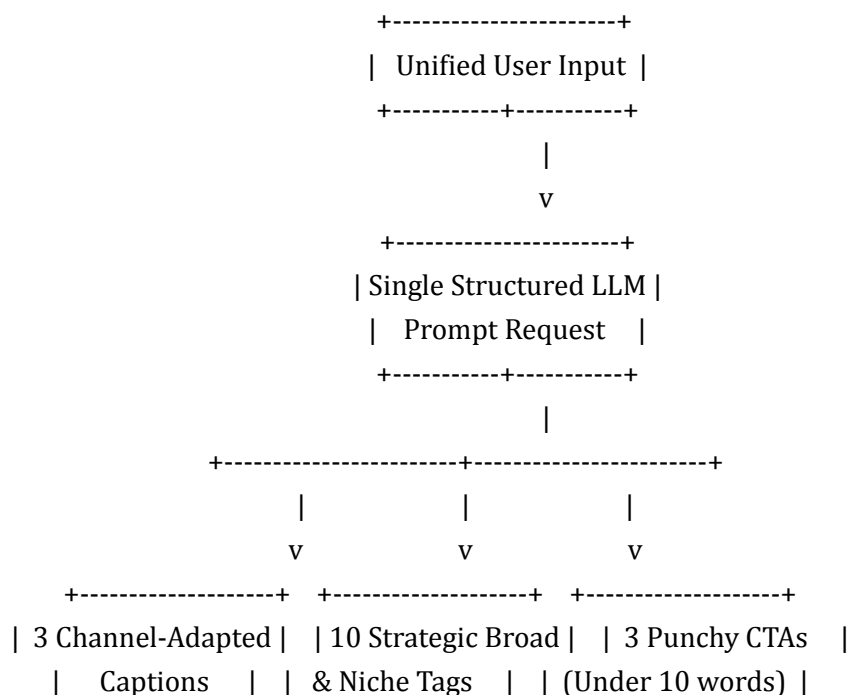
Milestone 2: Core Functionalities Development

Milestone Objective

Implement the fundamental content creation logic and backend workflows of CaptionAI. This involves creating a unified AI generation prompt that outputs all three core content types simultaneously (Captions, Hashtags, CTAs), establishing the Flask application structure, handling asynchronous input processing, and building fallback JSON parsing mechanisms to guarantee output integrity.

Activity 2.1: Develop Core Content Generation Features

Rather than making three separate API calls—which would increase latency and API usage—all three core social media deliverables are generated in a single request using a unified, engineered prompt strategy:



Module Specifications:

1. **Caption Generator Module:**
 - **Output:** 3 distinct, platform-optimized captions per generation request.
 - **Instagram Optimization:** Focuses on visual storytelling, relatable tone, and expressive emoji usage.
 - **LinkedIn Optimization:** Focuses on thought-leadership, value propositions, and professional insights.
2. **Hashtag Suggester Module:**
 - **Output:** Exactly 10 relevant hashtags balancing reach and relevance.
 - **Distribution:** 5 broad/popular high-volume tags for discovery + 5 micro/niche targeted tags for audience alignment.
 - **Validation:** Ensures every item includes the mandatory # prefix.
3. **Call-to-Action (CTA) Generator Module:**
 - **Output:** 3 concise, action-oriented phrases.
 - **Constraints:** Kept under 10 words per phrase.
 - **Purpose:** Tailored to trigger high-conversion behaviors (e.g., "Shop before it's gone!", "Book your free demo today").

Activity 2.2: Implement Flask Backend Logic

1. HTTP Route Definition & JSON Processing

The backend utilizes Flask to expose endpoints, accept AJAX payloads, and run boundary checks:

- **Endpoint Routing:** / serves the UI (`index.html`) while `/generate` accepts HTTP POST JSON payloads.
- **Payload Retrieval:** Extracted asynchronously using `request.get_json()`.
- **Input Validation Rules:**
 - Rejects empty or whitespace-only product/campaign descriptions.
 - Enforces a maximum input limit of **2,000 characters** to prevent prompt injection and token overflow.

2. Contextual Prompt Assembly

The server dynamically injects copywriting rules into the prompt using predefined configuration dictionaries:

- **TONE_DESCRIPTIONS:** Maps requested tones (**Professional**, **Casual**, or **Promotional**) to specific writing rules.
- **PLATFORM_TIPS:** Injects network constraints (e.g., 2,200 character limits for Instagram vs. 3,000 character limits for LinkedIn).

3. API Execution & Resilience Engine (`extract_json`)

To handle LLM output variations and prevent parsing failures, the backend processes API responses

through a multi-tier fallback parsing mechanism:

```
Python
def extract_json(text: str) -> dict:
    """Multi-pass parsing strategy to handle raw text and markdown-wrapped LLM
    outputs."""
    # Strategy 1: Direct JSON parse
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass

    # Strategy 2: Strip markdown code fences (`json ...`)
    match = re.search(r'```(?:json)?\s*([\\s\\S]*)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass

    # Strategy 3: Regex match for outermost curly brackets
    match = re.search(r'\{[\\s\\S]*\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass

    raise ValueError("Failed to extract a valid JSON structure from the AI response.")
```

Milestone 3: App.py Development

Milestone Objective

Construct and refine the complete application backend in `app.py`. This milestone focuses on building the primary routing structure, embedding copywriting prompt rules, establishing Groq API communications, implementing error-handling guards, and enforcing strict JSON output validation.

Activity 3.1: Writing the Main Application Logic (`app.py`)

1. Endpoints & Route Controllers

`app.py` defines two distinct endpoints to serve the application:

- **Home Route (GET /):** Renders the single-page frontend template `index.html`.
- **Generation Route (POST /generate):** Receives AJAX JSON payloads containing user inputs, validates fields, invokes the Groq API, and returns structured content to the frontend.

2. Copywriting Guidelines & Mappings

To guide the LLM's tone and formatting based on user selection, `app.py` uses pre-configured dictionary mappings:

- **TONE_DESCRIPTIONS:** Maps user-selected voices to copywriting rules.
 - `professional`: Formal, authoritative, and business-oriented.
 - `casual`: Friendly, conversational, and relatable with emojis.
 - `promotional`: Exciting, persuasive, urgency-driven with power words.
- **PLATFORM_TIPS:** Injects network-specific constraints.
 - `instagram`: Visually descriptive, storytelling focus (max 2,200 characters).
 - `linkedin`: Value-driven, thought-leadership focus (max 3,000 characters).

3. Structured Prompt Construction (`build_prompt`)

The application assembles a strict, zero-shot system instruction prompt enforcing schema boundaries:

Python

```
def build_prompt(product_info: str, platform: str, tone: str) -> str:
    tone_desc = TONE_DESCRIPTIONS.get(tone, TONE_DESCRIPTIONS["professional"])
    platform_tip = PLATFORM_TIPS.get(platform, PLATFORM_TIPS["instagram"])

    return f"""You are an expert social media content strategist and copywriter.
Generate social media marketing content based on the following context:
```

```
- Product/Campaign Info: {product_info}
- Target Platform: {platform.title()} ({platform_tip})
- Desired Tone: {tone.title()} ({tone_desc})
```

Respond strictly with a JSON object adhering to this structure:

```
{{
  "captions": ["Caption 1...", "Caption 2...", "Caption 3..."],
  "hashtags": ["#Tag1", "#Tag2", "#Tag3", "#Tag4", "#Tag5", "#Tag6", "#Tag7", "#Tag8", "#Tag9",
    "#Tag10"],
  "ctas": ["CTA 1", "CTA 2", "CTA 3"]
}}
```

Full Backend Implementation (`app.py`)

Python

```
import os
import re
```

```

import json
from flask import Flask, render_template, request, jsonify
from dotenv import load_dotenv
from groq import Groq

# Load environment variables
load_dotenv()

app = Flask(__name__)

# Initialize Groq client
api_key = os.environ.get("GROQ_API_KEY")
if not api_key:
    raise RuntimeError("GROQ_API_KEY environment variable is missing.")

client = Groq(api_key=api_key)

TONE_DESCRIPTIONS = {
    "professional": "formal, authoritative, and business-oriented. Use clear, concise language that conveys expertise.",
    "casual": "friendly, conversational, and relatable. Use informal language, emojis, and a warm tone.",
    "promotional": "exciting, persuasive, and action-driven. Use power words, urgency, and compelling offers."
}

PLATFORM_TIPS = {
    "instagram": "optimized for Instagram — visually descriptive, emotionally engaging. Max 2200 characters.",
    "linkedin": "optimized for LinkedIn — professional insights, value-driven style. Max 3000 characters."
}

def extract_json(text: str) -> dict:
    """Robust extractor to handle raw JSON or markdown-fenced outputs."""
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass

    match = re.search(r'```(?:json)?\s*([\\s\\S]*?)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass

    match = re.search(r'\{[\\s\\S]*\\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass

```

```
raise ValueError("Could not extract valid JSON from LLM response.")
```

```
@app.route("/")
```

```
def index():
```

```
    return render_template("index.html")
```

```
@app.route("/generate", methods=["POST"])
```

```
def generate():
```

```
    data = request.get_json() or {}
```

```
    product_info = data.get("product_info", "").strip()
```

```
    platform = data.get("platform", "instagram").lower()
```

```
    tone = data.get("tone", "professional").lower()
```

```
# Input Boundaries & Safeguards
```

```
if not product_info:
```

```
    return jsonify({"error": "Product information is required."}), 400
```

```
if len(product_info) > 2000:
```

```
    return jsonify({"error": "Input exceeds max character limit of 2,000."}), 400
```

```
try:
```

```
    prompt = build_prompt(product_info, platform, tone)
```

```
    chat_completion = client.chat.completions.create(
```

```
        messages=[
```

```
            {
```

```
                "role": "system",
```

```
                "content": "You are a social media content expert. Always respond with valid JSON only, no  
markdown formatting."
```

```
            },
```

```
            {
```

```
                "role": "user",
```

```
                "content": prompt
```

```
            }
```

```
        ],
```

```
        model="llama-3.3-70b-versatile",
```

```
        temperature=0.8,
```

```
        max_tokens=2048,
```

```
    )
```

```
    response_text = chat_completion.choices[0].message.content
```

```
    result = extract_json(response_text)
```

```
# Output Integrity Verification
```

```
if not all(k in result for k in ["captions", "hashtags", "ctas"]):
```

```
    raise ValueError("Response JSON missing required keys: 'captions', 'hashtags', or 'ctas'.")
```

```
return jsonify({
```

```
    "success": True,
```

```
    "captions": result["captions"],
```

```
    "hashtags": result["hashtags"],
```

```

        "ctas": result["ctas"],
        "platform": platform,
        "tone": tone
    })

except ValueError as ve:
    return jsonify({"error": f"Parsing Error: {str(ve)}"}), 500
except Exception as e:
    return jsonify({"error": f"Generation Failed: {str(e)}"}), 500

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5050, debug=True)

```

Milestone Deliverables Checklist

- [x] **Route Handling:** Configured `GET /` and `POST /generate` handlers.
- [x] **Validation Controls:** Enforced mandatory input checks and 2,000-character limits.
- [x] **Inference Parameters:** Bound Groq SDK calls to `llama-3.3-70b-versatile` with `temperature=0.8` and `max_tokens=2048`.
- [x] **Data Parsing:** Implemented `extract_json()` fallback extractor for strict JSON parsing.
- [x] **Schema Integrity:** Verified that output objects explicitly include `captions`, `hashtags`, and `ctas`.

Milestone 4: Frontend Development

Here is a clean, structured overview of **Milestone 4: Frontend Development** for your project documentation, report, or presentation.

Milestone 4: Frontend Development

Milestone Objective

Design and build an interactive, responsive, and aesthetically pleasing single-page user interface (SPA) for CaptionAI. This milestone focuses on implementing a modern **Dark Glassmorphism UI**, creating semantic HTML structures, writing modular CSS layouts, and developing dynamic JavaScript event handlers to manage asynchronous API interactions and DOM updates.

Activity 4.1: Designing and Developing the User Interface

1. Semantic HTML Structure (`templates/index.html`)

The user interface is built as a clean, accessible single-page layout divided into three main operational zones:

- **Brand Header:** Features the CaptionAI logo, wand magic icon (`Font Awesome`), and brand

tagline.

- **Input Panel:** A central frosted glass card containing a textarea for product details (up to 2,000 characters), platform toggle selectors (**Instagram / LinkedIn**), tone selectors (**Professional, Casual, Promotional**), and a prominent submission button.
- **Dynamic Results Grid:** A multi-column output container (initially hidden) configured to display generated captions on the left, alongside hashtag chips and CTA lists on the right.

2. Glassmorphism Styling & Responsive Layout (**static/css/style.css**)

- **Glassmorphism Aesthetic:** Implements translucent background layers (`rgba(255, 255, 255, 0.05)`), subtle light borders, and hardware-accelerated blur filters (`backdrop-filter: blur(16px)`).
- **Ambient Animation:** Uses fixed animated gradient background blobs (`blob-1, blob-2, blob-3`) with continuous floating keyframe animations to create depth.
- **Adaptive Grids & Flexbox:** Organizes the primary layout using CSS Grid and Flexbox, with media queries ensuring responsiveness across desktop, tablet, and mobile displays.

3. Modular Output Components

- **Caption Cards:** Individual frosted cards rendering generated copy alongside a one-click **Copy to Clipboard** button that provides immediate visual feedback (button turns green with a checkmark for 2 seconds).
- **Hashtag Chips:** Compact pill-shaped badges for each generated hashtag, complete with automated `#` prefix formatting.
- **CTA List Items:** Cleanly formatted list components displayed inside a glass card with an accompanying bullhorn icon header.

Activity 4.2: Dynamic Content Rendering (**static/js/main.js**)

1. Asynchronous Form Submission

Attaches an event listener to the generation form to intercept default browser submissions and execute asynchronous **HTTP POST** requests using the native JavaScript **Fetch API**:

JavaScript

```
document.getElementById('generator-form').addEventListener('submit', async (e) => {  
  e.preventDefault();
```

```
  // UI Loading State
```

```
  const submitBtn = document.getElementById('submit-btn');
```

```
  submitBtn.disabled = true;
```

```
  submitBtn.innerHTML = '<span class="spinner"></span> Generating...';
```

```
  const payload = {
```

```
    product_info: document.getElementById('product_info').value,
```

```
    platform: document.querySelector('input[name="platform"]:checked').value,
```

```
    tone: document.querySelector('input[name="tone"]:checked').value
```

```
  };
```

```

try {
  const response = await fetch('/generate', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });

  const data = await response.json();
  if (data.success) {
    renderResults(data);
  } else {
    alert(data.error || 'Generation failed.');
```

```

  }
} catch (err) {
  console.error('Error:', err);
  alert('An unexpected error occurred.');
```

```

} finally {
  // Restore Button State
  submitBtn.disabled = false;
  submitBtn.innerHTML = 'Generate Content <i class="fas fa-magic"></i>';
}
});

```

2. DOM Injection & Auto-Scroll (**renderResults**)

Dynamically constructs HTML elements for captions, hashtags, and CTAs, inserts them into their designated DOM containers, reveals the results container, and smooth-scrolls the viewport to the output section:

JavaScript

```

function renderResults(data) {
  const resultsSection = document.getElementById('results-section');
  const captionsContainer = document.getElementById('captions-container');
  const hashtagsContainer = document.getElementById('hashtags-container');
  const ctasContainer = document.getElementById('ctas-container');

  // Clear previous results
  captionsContainer.innerHTML = "";
  hashtagsContainer.innerHTML = "";
  ctasContainer.innerHTML = "";

  // Render Captions
  data.captions.forEach((caption, index) => {
    const card = document.createElement('div');
    card.className = 'caption-card';
    card.innerHTML = `
      <p>${caption}</p>
      <button class="copy-btn" onclick="copyToClipboard('${escapeJs(caption)}', this)">
        <i class="far fa-copy"></i> Copy
    `
  });
}

```

```

        </button>
    `;
    captionsContainer.appendChild(card);
  });

  // Render Hashtags as Chips
  data.hashtags.forEach(tag => {
    const chip = document.createElement('span');
    chip.className = 'hashtag-chip';
    chip.textContent = tag.startsWith('#') ? tag : `#${tag}`;
    hashtagsContainer.appendChild(chip);
  });

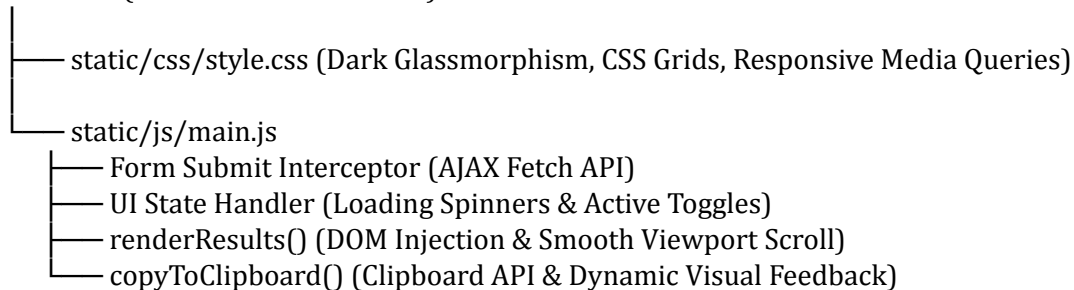
  // Render CTAs
  data.ctas.forEach(cta => {
    const li = document.createElement('li');
    li.textContent = cta;
    ctasContainer.appendChild(li);
  });

  // Reveal and Smooth Scroll
  resultsSection.classList.remove('hidden');
  resultsSection.scrollIntoView({ behavior: 'smooth' });
}

```

Frontend Architecture Overview

index.html (Semantic SPA Structure)



Milestone Deliverables Checklist

- [x] **Semantic HTML Layout:** Created single-page template with header, input form, and result containers.
- [x] **Glassmorphism CSS Theme:** Implemented frosted glass panels, animated background gradients, and responsive grid layouts.
- [x] **Asynchronous Data Exchange:** Connected the frontend to `/generate` via JavaScript Fetch API.
- [x] **Dynamic Component Rendering:** Programmed automated DOM generation for caption cards, hashtag chips, and CTA lists.
- [x] **User Experience Utilities:** Added interactive copy-to-clipboard functionality, loading spinners,

and automatic scroll-to-view navigation.

Milestone 5: Deployment

Here is a clean, structured overview of Milestone 5: Deployment & Public Exposure for your project documentation, report, or presentation.

Milestone 5: Deployment & Integration

Milestone Objective

Transition CaptionAI from a local development script to a accessible application. This milestone covers configuring the local WSGI web server, establishing secure public URL tunneling using Ngrok, orchestrating the application startup flow, and executing end-to-end integration testing.

Activity 5.1: Configure Local Server Execution

The application uses Flask's built-in WSGI development server for execution, configured to run on a dedicated local port:

- Host Binding: `127.0.0.1` (localhost)
- Port Allocation: `5050` (selected to avoid common port conflicts with default services on 5000/8080)
- Environment Separation: Maintained separate launch configurations for local development (`app.py` with `debug=True`) and public tunneling (`run_public.py` with `debug=False`).

Activity 5.2: Public Exposure with Ngrok Tunneling (`run_public.py`)

To allow external users or remote devices to access the locally running application over secure HTTPS without manual router port-forwarding, CaptionAI integrates Ngrok via the `pyngrok` Python wrapper.

Public Launcher Implementation (`run_public.py`)

Python

import os

import sys

from dotenv import load_dotenv

from pyngrok import ngrok

from app import app

Load environment variables

load_dotenv()

```

# Retrieve optional authtoken from .env
ngrok_authtoken = os.environ.get("NGROK_AUTHTOKEN")
if ngrok_authtoken:
    ngrok.set_auth_token(ngrok_authtoken)

PORT = 5050

def start_server():
    try:
        # Establish HTTP tunnel on port 5050
        public_url = ngrok.connect(PORT).public_url
        print("\n" + "=" * 60)
        print(f"🚀 CAPTIONAI IS LIVE AND PUBLICLY ACCESSIBLE!")
        print(f"🔗 Public URL: {public_url}")
        print(f"🏠 Local URL: http://127.0.0.1:{PORT}")
        print("=" * 60 + "\n")

        # Run Flask app with debug=False to prevent duplicate tunnel creation
        app.run(host="127.0.0.1", port=PORT, debug=False)

    except Exception as e:
        print(f"❌ Failed to start Ngrok tunnel or Flask app: {e}", file=sys.stderr)
        sys.exit(1)

if __name__ == "__main__":
    start_server()

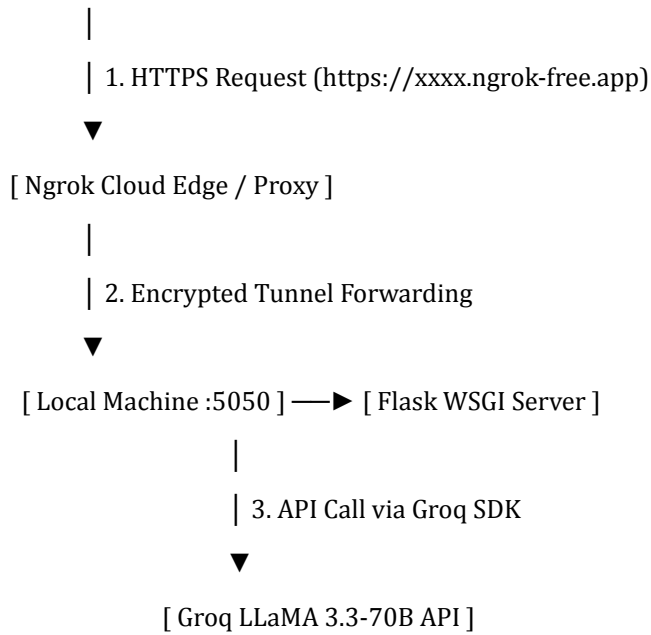
```

Why `debug=False` is Enforced:

When Flask runs with `debug=True`, its automatic reloader process spawns two separate Python processes. In a tunneling environment, this causes `pyngrok` to attempt creating duplicate tunnels on the same port, resulting in port binding errors and session crashes. Disabling debug mode in `run_public.py` ensures a single, stable process execution.

Activity 5.3: End-to-End Execution Flow & Architecture

[External User / Mobile Device]



Launch Sequence:

- 1. Command Execution: Run `python run_public.py` in the activated virtual environment.
- 2. Environment Loading: `.env` variables (`GROQ_API_KEY`, optional `NGROK_AUTHTOKEN`) are loaded into memory.
- 3. Tunnel Creation: `pyngrok` requests a public secure endpoint from Ngrok servers and routes traffic to `127.0.0.1:5050`.
- 4. App Initialization: Flask initializes routes (`/` and `/generate`) and begins listening for inbound web traffic.
- 5. Runtime Telemetry: Terminal displays both local and public URLs for verification.

Activity 5.4: Integration & System Verification Testing

Test Case	Scenario / Input	Expected Result	P a s s / F a i l
Local Access	Navigate to [<code>http://127.0.0.1:5050</code>] (<code>http://127.0.0.1:5050</code>)	Glassmorphism UI renders correctly with all controls	P A S S
Public Tunneling	Navigate to generated <code>https://*.ngrok-free.ap</code> p URL	Public site renders seamlessly on remote/mobile browsers	P A S S

Generation Flow	Submit product brief, select LinkedIn + Professional	Asynchronously returns 3 captions, 10 hashtags, and 3 CTAs	PAS
Clipboard Copy	Click "Copy" on any generated caption card	Copy text updates system clipboard and button shows green checkmark	PAS
Validation Check	Submit empty brief or text > 2000 characters	System gracefully displays error banner without crashing server	PAS

Milestone Deliverables Checklist

- [x] WSGI Configuration: Configured Flask server binding on `127.0.0.1:5050`.
- [x] Tunneling Wrapper: Created `run_public.py` leveraging `pyngrok` for instant public HTTPS URLs.
- [x] Process Isolation: Disabled Flask auto-reloader (`debug=False`) to guarantee tunnel stability.
- [x] End-to-End Testing: Conducted successful integration testing across local and remote devices.

Exploring the Website’s Web Pages:

CaptionAI is constructed as an interactive Single-Page Application (SPA). Rather than reloading separate HTML pages for different functions, all user interactions, input handling, and content generation occur dynamically within a single, seamless dashboard (`index.html`).

1. Primary Page Structure (`templates/index.html`)

The frontend layout is organized into three core functional zones:

+-----+										
	BRAND HEADER									
	Logo • Magic Wand Icon • Tagline & Subtitle									
+-----+										
	INPUT PANEL									
	[Product / Campaign Brief Textarea (Max 2,000 Chars)								]	
	[Platform Selector: Instagram LinkedIn								]	
	[Tone Selector: Professional Casual Promotional								]	
	[Action Button: "Generate Content ✨"								]	
+-----+										
	DYNAMIC RESULTS SECTION (Hidden)									
	+-----+				+-----+					
	GENERATED CAPTIONS GRID					HASHTAGS & CTAS PANEL				
	• Caption Card 1 [Copy Btn]					• 10 Hashtag Chips				
	• Caption Card 2 [Copy Btn]					• 3 Call-to-Action Items				
	• Caption Card 3 [Copy Btn]									
	+-----+				+-----+					
+-----+										

2. Interactive Page Components & States

A. The Input Form Zone

- Product Brief Input: A auto-resizing text area where users enter campaign details (enforced limit of 2,000 characters).
- Platform Toggle Pills: Custom radio button groups allowing users to toggle target network constraints (Instagram vs. LinkedIn).
- Tone Radio Grid: Interactive selectors to set the target copywriting voice (Professional, Casual, or Promotional).
- Interactive Submission Button: Trigger button that dynamically morphs into a loading spinner ("**Generating...**") during API communication to prevent duplicate submissions.

B. The Results Display Zone

Once the backend processes the Groq API output, the frontend dynamically reveals the hidden `#results-section` and smooth-scrolls into view:

1. Caption Stream (Left Column): Renders 3 individual, styled card containers displaying formatted post text along with a one-click Copy to Clipboard button that provides real-time UI feedback (changes to green with a checkmark upon click).
2. Hashtags Cloud (Top Right): Renders 10 interactive pill-shaped badges (`.hashtag-chip`) formatted automatically with mandatory `#` prefixes.
3. CTA Checklist (Bottom Right): Displays 3 punchy, high-conversion action prompts inside a clean glass card with an icon header.

3. Asynchronous Page Navigation Flow (`main.js`)

Since there are no multi-page redirects, user navigation relies on Asynchronous DOM State Transitions:

1. Initial State: The user sees the Brand Header and Input Panel. The Results Section is hidden (`.hidden` class applied).
2. Loading State: Upon submitting the form, `main.js` catches the event, disables inputs, displays a loading spinner on the submit button, and sends a `POST` request to `/generate`.
3. Render State: Upon receiving the JSON response, JavaScript injects the generated strings directly into designated DOM nodes (`#captions-container`, `#hashtags-container`, `#ctas-container`), removes the `.hidden` class, and triggers `scrollIntoView({ behavior: 'smooth' })`.
4. Error State: If validation fails (e.g., empty brief or API error), a alert banner or modal notifies the user without losing their entered text.

Conclusion:

The development and deployment of **CaptionAI** successfully demonstrate the power of combining modern web frameworks with ultra-low-latency Large Language Model (LLM) inference. By leveraging a modular **Three-Tier Architecture**, the platform seamlessly bridges an intuitive user interface with Groq's high-performance AI infrastructure to solve a common marketing challenge: generating high-converting, platform-tailored social media content in seconds.